

Tarefa 1

1. Introdução

O desempenho de programas computacionais está intimamente relacionado à eficiência com que os dados são acessados na hierarquia de memória. Em operações sobre matrizes de grande porte, o padrão de acesso aos elementos, sequencial ou com saltos, determina em que medida o hardware pode aproveitar a *memória cache*, impactando diretamente o tempo de execução.

Este relatório investiga experimentalmente o efeito do padrão de acesso à memória no contexto da operação de multiplicação matriz-vetor (MxV). Duas implementações em linguagem C foram desenvolvidas e comparadas: uma que percorre a matriz por **linhas** (laço interno variando a coluna) e outra por **colunas** (laço interno variando a linha). Os resultados são analisados à luz dos conceitos de localidade espacial, *cache miss* e hierarquia de memória (L1, L2 e L3).

2. Fundamentação Teórica

2.1 Hierarquia de Memória

Processadores modernos operam em frequências muito superiores à velocidade de acesso à memória principal (RAM). Para mitigar essa disparidade, adota-se uma hierarquia de memórias com diferentes níveis de capacidade e latência:

- **Registradores:** acesso em menos de 1 ciclo; capacidade da ordem de dezenas de bytes.
- **Cache L1:** latência de 1–4 ciclos; capacidade típica de 32–64 KB por núcleo.
- **Cache L2:** latência de 10–20 ciclos; capacidade de 256 KB a 1 MB por núcleo.
- **Cache L3:** latência de 30–60 ciclos; capacidade de 4–64 MB, compartilhada entre núcleos.
- **Memória RAM:** latência de 60–200 ns; capacidade de gigabytes.

Quando um dado não se encontra em nenhum nível de cache (*cache miss*), é necessário buscá-lo na RAM, incorrendo em penalidade de dezenas a centenas de ciclos.

2.2 Localidade de Referência

O desempenho da cache depende fundamentalmente do princípio de **localidade de referência**, que se manifesta de duas formas:

- **Localidade temporal:** dados acessados recentemente tendem a ser reutilizados em breve.
- **Localidade espacial:** o acesso a um dado aumenta a probabilidade de que endereços próximos sejam acessados em seguida. Ao carregar um dado, a cache traz consigo uma *cache line* inteira (tipicamente 64 bytes), aproveitando essa propriedade.

2.3 Armazenamento de Matrizes em C (Row-Major Order)

Em linguagem C, matrizes bidimensionais são armazenadas em memória em *row-major order*: os elementos de uma mesma linha ocupam posições contíguas. Formalmente, para uma matriz $A[N][N]$, o endereço do elemento $A[i][j]$ é dado por:

$$\text{addr}(A[i][j]) = \text{addr}(A[0][0]) + (i \times N + j) \times \text{sizeof}(\text{int})$$

Consequentemente, acessar elementos ao longo de uma linha é sequencial; acessar ao longo de uma coluna implica saltos de $N \times \text{sizeof}(\text{int})$ bytes entre cada acesso.

3. Metodologia

3.1 Implementação

O experimento foi implementado em linguagem C com a seguinte estrutura:

- **mxv_linhas(N):** laço externo sobre linhas i , laço interno sobre colunas j . Acessa $A[i][j]$ sequencialmente na memória.
- **mxv_colunas(N):** laço externo sobre colunas j , laço interno sobre linhas i . Acessa $A[i][j]$ com stride de $N \times 4$ bytes.
- **Medição de tempo:** função `clock_gettime(CLOCK_MONOTONIC)`, com resolução nanosegundo.
- **Tamanhos testados:** $N = 100, 200, 300, \dots, 2000$ (incremento de 100).
- **Inicialização:** todos os elementos da matriz e do vetor definidos como 1, eliminando variações por dados.

3.2 Ambiente de Execução

Os testes foram executados em ambiente Linux com compilador GCC, sem otimizações de alto nível (flags padrão), garantindo que o comportamento observado reflita diretamente os padrões de acesso à memória e não transformações automáticas do compilador.

4. Resultados Experimentais

A Tabela 1 apresenta os tempos de execução medidos (em segundos) para ambas as versões, bem como a diferença percentual relativa à versão por linhas. Células destacadas em amarelo correspondem a diferenças superiores a 50%.

Tabela 1 — Tempos de execução das versões por linhas e por colunas.

N	Linhas (s)	Colunas (s)	Diferença (%)
100	0,000007	0,000008	14,3%
200	0,000028	0,000030	7,1%
300	0,000065	0,000068	4,6%
400	0,000115	0,000148	28,7%
500	0,000179	0,000209	16,8%
600	0,000243	0,000336	38,3%
700	0,000352	0,000482	37,0%
800	0,000477	0,000603	26,4%
900	0,000565	0,000738	30,6%
1000	0,000753	0,000898	19,3%
1100	0,000535	0,001014	89,5%
1200	0,001036	0,001382	33,4%
1300	0,000825	0,001419	72,0%
1400	0,000967	0,001665	72,2%
1500	0,000906	0,002095	131,3%
1600	0,001587	0,002251	41,8%
1700	0,001943	0,002691	38,5%
1800	0,002201	0,003273	48,7%
1900	0,001443	0,003383	134,5%
2000	0,002579	0,004553	76,5%

Fonte: elaborado pelo autor.

5. Análise e Discussão

5.1 Comportamento para Matrizes Pequenas ($N \leq 700$)

Para $N \leq 700$, as diferenças de tempo são pequenas e, em alguns casos, dentro da margem de variação natural das medições. Uma matriz 700×700 de inteiros (4 bytes) ocupa:

$$700^2 \times 4 \text{ bytes} = 1.960.000 \text{ bytes} \approx 1,87 \text{ MB}$$

Esse volume ainda pode ser parcialmente acomodado nas caches L2 e L3 do processador. Com a maior parte dos dados nas caches, os acessos não sequenciais da versão por colunas ainda encontram os elementos necessários sem recorrer à RAM com frequência, resultando em desempenho similar ao da versão por linhas.

5.2 Divergência para Matrizes Maiores ($N \geq 1000$)

A partir de $N \approx 1000$, a diferença entre os tempos torna-se consistente e crescente. Uma matriz 1000×1000 ocupa:

$$1000^2 \times 4 \text{ bytes} = 4.000.000 \text{ bytes} = 4 \text{ MB}$$

Nesse ponto, a matriz começa a exceder a cache L3 típica (ou ao menos a cache L2). Com os dados não cabendo mais inteiramente em cache, o comportamento dos dois padrões de acesso diverge significativamente:

- **Versão por linhas:** cada *cache line* carregada traz vários elementos úteis consecutivos de $A[i][*]$. O número de *cache misses* é da ordem de $N^2/16$ (assumindo ints de 4 bytes e cache lines de 64 bytes).
- **Versão por colunas:** o stride entre acessos consecutivos $A[i][j]$ e $A[i+1][j]$ é de $N \times 4$ bytes. Para $N = 2000$, esse stride é de 8 KB — muito maior que uma cache line — resultando em um *cache miss* quase a cada acesso e atingindo a RAM continuamente.

5.3 Resultado para $N = 2000$

No maior caso testado, $N = 2000$, os tempos obtidos foram:

- **Versão por linhas:** 0,002579 s
- **Versão por colunas:** 0,004553 s

A versão por colunas foi **aproximadamente 76,5% mais lenta**, evidenciando o custo elevado dos *cache misses* quando a matriz supera a capacidade das caches e os acessos são realizados com grandes saltos de memória.

6. Conclusão

O experimento demonstrou de forma clara e quantitativa como o padrão de acesso à memória influencia o desempenho de programas intensivos em dados. A versão que percorre a matriz por linhas, explorando a localidade espacial garantida pelo armazenamento *row-major* da linguagem C, apresentou desempenho consistentemente superior à versão por colunas, especialmente para matrizes de grande porte.

Os resultados experimentais corroboram a teoria da hierarquia de memória: enquanto os dados cabem nas caches, as diferenças são marginais; à medida que o volume de dados excede a capacidade das caches L2 e L3, a penalidade por *cache misses* na versão por colunas torna-se dominante e cresce de forma progressiva.

Do ponto de vista prático, este experimento evidencia que **a localidade espacial é um fator fundamental para o desenvolvimento de algoritmos de alto desempenho**. Projetistas de software e de algoritmos devem considerar a organização dos dados na memória ao estruturar laços que operam sobre matrizes ou outros dados multidimensionais, pois a escolha do padrão de acesso pode resultar em diferenças de desempenho da ordem de dezenas a mais de 100%.

7. Código Fonte

```
1  #include <stdio.h>
2  #include <time.h>
3
4  #define MAXN 2000
5  #define BILLION 1000000000.0
6
7  int A[MAXN][MAXN];
8  int x[MAXN];
9  int y[MAXN];
10
11 void init(int N){
12     for(int i=0;i<N;i++){
13         x[i] = 1;
14         for(int j=0;j<N;j++){
15             A[i][j] = 1;
16         }
17     }
18
19     /* Acesso por LINHAS */
20     void mxv_linhas(int N){
21         for(int i=0;i<N;i++){
22             y[i] = 0;
23             for(int j=0;j<N;j++){
24                 y[i] += A[i][j] * x[j];
25             }
26         }
27
28         /* Acesso por COLUNAS */
29         void mxv_colunas(int N){
30             for(int i=0;i<N;i++){
31                 y[i] = 0;
32
33                 for(int j=0;j<N;j++){
34                     for(int i=0;i<N;i++){
35                         y[i] += A[i][j] * x[j];
36                     }
37                 }
38             }
39             /* Medição de tempo */
40             double medir(void (*func)(int), int N){
41                 struct timespec ini, fim;
42                 clock_gettime(CLOCK_MONOTONIC, &ini);
43
44                 func(N);
45
46                 clock_gettime(CLOCK_MONOTONIC, &fim);
47
48                 return (fim.tv_sec - ini.tv_sec) +
49                     (fim.tv_nsec - ini.tv_nsec) / BILLION;
50             }
51
52             int main(){
53                 printf("N\tLinhas(s)\tColunas(s)\n");
54
55                 for(int N=100; N<=2000; N+=100){
56                     init(N);
57
58                     double t1 = medir(mxv_linhas, N);
59                     double t2 = medir(mxv_colunas, N);
60
61                     printf("%d\t%.6f\t%.6f\n", N, t1, t2);
62                 }
63
64                 return 0;
65             }
```